

CATEGORIZING ML ALGORITHMS FOR NETWORK BASED ATTACKS

Aniket Nare¹, Rishabh Chaddha¹

¹Purdue University

e-mail: anare@purdue.edu, rchaddha@purdue.edu

Abstract – With the increasing number of people joining the internet, the number of attacks is also increasing. There are two broad methods to detect attacks on the internet - signature-based and anomaly detection based. Of the two, anomaly detection techniques are preferred due to their speed, efficiency, and ability to even detect zero-day attacks. Anomaly detection assumes the premise that the attacker will have a behavior different than that of a benign user. Also, the anomaly detection techniques aim to unearth both already known and potentially new attacks. Due to the mentioned reasons, machine learning techniques seem suitable for anomaly detection. In this work, the efficiency of commonly used machine learning algorithms (Naive Bayes, Random Forest, Decision Tree, Adaboost, Neural Networks, and Nearest Neighbor) have been compared in detecting attacks using a dataset containing features related to network-based attacks (on the CSE-CIC-IDS2018 dataset). Then, the best performing algorithm is taken and its robustness is evaluated in adversarial conditions.

Keywords – Adaboost, Adversarial Machine Learning, Decision Tree, kNN, Naive Bayes, Network Based Anomaly Detection, Neural Networks, Random Forest.

I. INTRODUCTION

The users of services provided by the internet have been increasing at an exponential rate. The rise in the number of users has led to a commensurate rise in the number of attackers. Network traffic can be analyzed to find out the cause of attacks that have taken place and to detect any attacks in the future. Two basic methods are widely employed to detect attacks. These are signature-based methods and network anomaly detection methods.

Signature-based methods create a database and keep updating it as and when they receive new attack information. Then they use these databases to detect attacks. This poses some problems. Firstly, the database maintained by these methods needs to be updated every time new attack information is available. The second problem is that if the databases do not contain any particular attack (as in the case of a new kind of attack), then these signature-based methods cannot detect the said kind of attack. Another problem is that the internet today mostly uses SSL or TLS encrypted traffic. Signature-based attacks do not work well with this type of data because these methods cannot observe the contents of network packets.

Network anomaly detection deals with the task of finding rare and unusual patterns in data obtained from network traffic. It can determine how anomalous network behavior is related to network attacks. One of the attractive features of network

anomaly detection systems is that they can be used to find out attacks that have never been carried out before. Such systems can also detect attacks in an SSL/TLS setting as they do not need to inspect network packet elements. They just observe general network properties like packet size, connection time, flow, number of packets on either side of the stream, and so on.

Anomalies can be classified into three types:

- **Point anomalies:** In this case, a single data point shows behavior that differs from all the other points in the dataset. As an example, consider a person whose daily spend is \$50, but on a particular day, the mentioned person spends \$1000.
- **Contextual anomalies:** This takes place when deviation takes place due to some underlying condition that is different from the normal. As an example, a person spends \$500 on a particular day to buy a birthday gift for someone.
- **Collective anomaly:** This takes place when a group of points deviates from the normal. As an example, many people spend more than their average daily spend on Christmas.

These types of anomalies in network traffic can be used to detect attacks in the network [1]. Network attacks can be classified into the following:

- **Denial of Service:** In this type of attack, asymmetry in the system is exploited by an attacker to make resources for a service unavailable to legitimate users. An attacker can aim to exploit the majority of a system's resources so that the system becomes unavailable to requests.
- **User to Root (U2R):** In this kind of attack, an attacker enters the system as a normal user and can gain an access to a higher (root or admin) privilege by performing certain attacks.
- **Remote to User (R2U):** In this kind of attack, an attacker infiltrates the network in which the target machine is present and then can get hold of an administrator account to get access to the system's resources. Basically, the adversary enters the system and gets hold of a system.
- **Probe (Information Gathering):** In this kind of attack, information can be collected about the target, and then this information can be exploited by carrying out some other kind of attack.

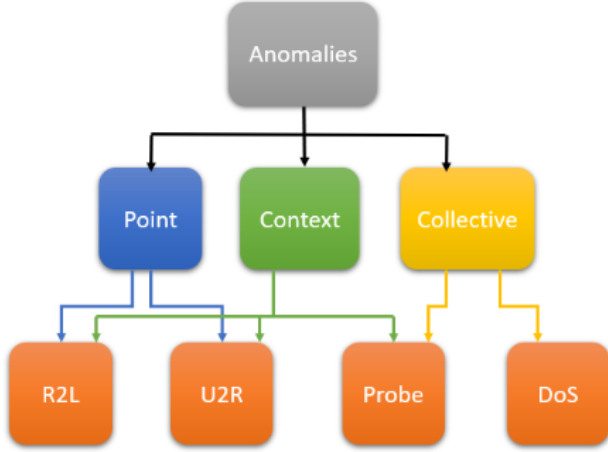


Fig. 1. Relation between type of anomalies and attacks

Because anomaly detection methods aim to find certain patterns in network traffic that might lead to a never before seen behavior, machine learning techniques are mostly used to design anomaly detection models. Moreover, Figure 1 summarizes how the types of anomalies and the types of attacks are interconnected. In case of network attack detection, machine learning models can observe the kind of anomalies leading to an attack and can predict the occurrence of an attack in any new dataset. Machine learning models give varying accuracy depending on the type of data they operate on. In this project, we compare Naive Bayes, Random Forest, Decision Tree, Adaboost, Neural Networks and Nearest Neighbor machine learning models to see which performs best on a network data (the dataset chosen for this purpose is CSE-CIC-IDS-2018).

Study indicates that attacks against machine learning algorithms are also possible [2]. The dataset used to train the machine learning model, for example, can be perturbed in such a way that data entries classified as attacks are misclassified to that of a benign item. As a result, even the high accuracy trained model can be tricked, resulting in incorrect predictions on testing data. Such attacks are classified as adversarial machine learning, in which the accuracy of machine learning models is affected in some way. We hope to demonstrate how machine learning algorithms used for anomaly detection respond in such challenging situations in this study.

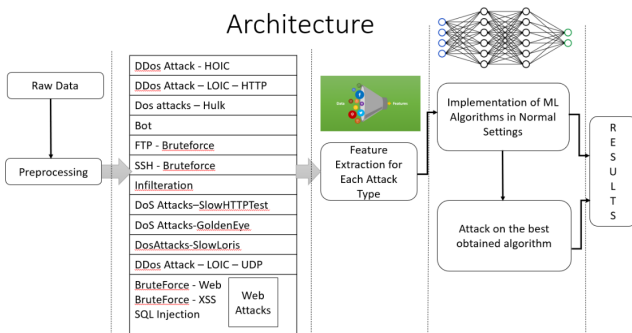


Fig. 2. Architecture for comparing machine learning algorithms

For machine learning algorithms to perform effectively,

one of the most critical phases is preprocessing. The data in the dataset will be cleansed during the preparation stage. As a result, certain records with trash values will be erased. Furthermore, the majority of the data is transformed to categorical data so that machine learning models can deal with numbers. This phase is critical because certain records with obviously incorrect values might cause overfitting in the machine learning model, which is when the model performs well on the training data but badly on the test data.

Subsequently, in the Attack Filtering stage, the data is divided into 11 files, one for each type of attack. Since there are more than 80 features making it complex for training any Machine Learning model, we use the Random Forest Regressor to extract the most important features (Table I) on each kind of attack. These features are then used to perform Machine Learning training in normal and testing in both normal and adversarial settings.

Features	Description
fl_dur	Flow Duration
Bw_pkt_l_avg	Mean size of packet in backward direction
fl_pkt_s	flow packets rate that is number of packets transferred per second
tot_fw_pk	Total packets in the forward direction
tot_l_fw_pkt	Total size of packet in forward direction
fw_iat_max	Maximum time between two packets sent in the forward direction
fw_pkt_l_std	Standard deviation size of packet in forward direction

TABLE I

Important features to detecting an attack

II. LITERATURE SURVEY

In this section, we stated, in brief, the observations noticed by reviewing various datasets published in the Network-based Anomaly detection field over the period of time. Due to privacy concerns, datasets used for network packet analysis in commercial products are not readily available. There are a few publicly available datasets that are extensively used as benchmarks, such as mentioned in this section. Moreover, we examine research papers that use the CIC-CSE-IDS-2018 dataset and draw out the shortcomings. Mainly focusing on the Machine Learning algorithms used in order to select algorithms best suited for training the mentioned dataset. Finally, we have mentioned our discovery about adversarial machine learning and conclusions drawn from related work.

A. Datasets

A substantial volume of malicious and benign network traffic is required for training and testing processes in the identification of network anomalies using machine learning algorithms. Real network traffic, on the other hand, cannot be utilized publicly due to privacy concerns. Many datasets have been created and are still being created to suit this need. This section will provide information on various common datasets, which will then be compared and analyzed to choose which ones to employ in the implementation phase.

1) *DARPA [3]*: The goal of this dataset, which was generated with DARPA support by the MIT Lincoln Laboratory, is to establish a training and testing environment for Intrusion Detection Systems. These datasets are out-of-date since they don't include information on new malware attacks. In diverse network topologies, operating systems,

software, and criminal toolkits, for example, attackers behave differently.

2) *NSL-KDD [4]*: It is a publicly available dataset that evolved from the KDD cup99 dataset. A statistical study of the cup99 dataset revealed significant flaws that have a significant impact on intrusion detection accuracy and contribute to a skewed assessment of NIDS [5].

A large number of duplicate packets in the KDD data set is the key issue. The KDD training and test sets were investigated in this research, and it was discovered that about 78 percent and 75 percent of network packets are repeated in both the training and testing datasets [5].

Due to a large number of duplicate instances in the training set, machine-learning algorithms will be biased towards normal instances, preventing them from learning irregular cases, which are often more harmful to the computer system.

3) *ISCX 2012*: Despite the fact that various datasets have been utilized to discover anomalies, complaints have emerged since these data have become out-of-date or the synthetic data employed does not match real-world data. To address these issues, ISCX 2012 (Intrusion detection evaluation dataset) was established in 2012 utilizing a seven-day Internet stream on the Canadian Institute for Cybersecurity's testbed.

Real network traffic traces were studied in this dataset to determine normal behavior for computers from HTTP, SMTP, SSH, IMAP, POP3, and FTP protocol traffic [6]. This dataset is built on tagged, actual network traffic that includes a variety of attack scenarios.

The ISCX 2012 data set, on the other hand, excludes SSL / TLS (Traffic Sockets Layer / Transport Layer Security) traffic, which accounts for more than half of today's Internet traffic, implying that it will be insufficient to satisfy current demands.

4) *CSE-CIC-IDS-2018 [7]*: The dataset was published in a joint project by the Communications Security Establishment (CSE) and the Canadian Institute of Cybersecurity (CIC), and the newest version of the intrusion detection dataset, CSE-CIC-IDS2018, was released in 2018. The upgraded version has a similar structure as CICIDS2017 and has a class imbalance.

CSE-CIC-IDS-2018, on the other hand, was built from a significantly bigger network of simulated client targets and attack computers, yielding a dataset with 16,233,002 instances acquired from 10 days of network traffic.

Attack traffic accounts for about 17% of all instances. Refer to Figure 3. The percentage distribution for the seven categories of network traffic covered by CSE-CIC-IDS-2018 is shown in Table II. The data is broken down into 10 CSV files that may be downloaded from AWS. There are 79 separate features in nine files, and 83 independent features in the remaining file.

The data has been categorized by day. Each day contains the raw data from each computer, including network traffic (Pcaps) and event logs (Windows and Ubuntu event logs). The CICFlowMeter-V3 extracts more than 80 traffic features from the raw data and stores them as a CSV file per machine during the feature extraction procedure.

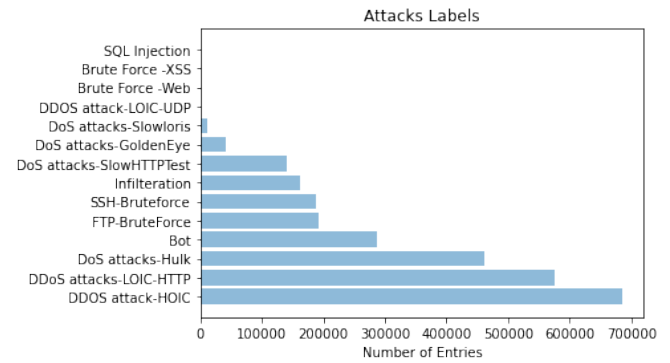


Fig. 3. Distribution of attack entries in the stream records of CSE-CIC-IDS-2018 dataset. Since there is a class imbalance, we grouped few classes under a general class. For instance, SQL, XSS and Web were grouped under Web Attacks class.

Traffic Type	Distribution (%)
Benign	83.070
DDoS	7.786
DoS	4.031
Brute force	2.347
Botnet	1.763
Infiltration	0.997
Web attacks	0.006

TABLE II
Network Traffic Distribution

B. Machine Learning

In this project, supervised learning methods have been employed to make use of a well-labeled dataset that has been manually labeled. It is hoped to gain a high-performance advantage without incurring a cost disadvantage in this regard.

Naive Bayes, Random Forest, Decision Tree (ID3), AdaBoost, MLP, and K Nearest Neighbors are some of the machine learning algorithms employed in the application phase. When selecting these approaches, the goal is to combine the most frequently used algorithms as mentioned in the Table for training such datasets, with various inherent qualities. The methods employed in this context are reviewed in-depth as follows:

1) *Naive Bayes*: A network of probabilities called Naive Bayes consists of a parent node that represents the unseen state and many child nodes that represent the observed states.

The child nodes in this statistical network are presumed to be independent of one another. Although the Naive Bayes theory is based on this notion, the assumption that sub-nodes are independent of one another is frequently incorrect. The Naive Bayes approach achieves poorer accuracy when compared to other machine learning algorithms because of this [8]. Despite this shortcoming, it is one of the most popular machine learning approaches due to its quick training duration and low computing cost.

2) *Random Forest*: From the data in the training set, random forest constructs N decision trees. It resamples the training set for each tree at random during this procedure. As a result, n decision trees are generated, each of which is unique.

Finally, voting is carried out by picking new estimates from N tree estimations. The final value is decided by the value with the highest rating.

Random Forest, because of its unique inherent qualities, brings many advantages. For instance, this method is capable of handling very huge and complex datasets. The overfitting issue that plagues decision trees is uncommon with this approach. It may be used to solve a wide range of machine learning problems. It's also a good idea to deal with the data set's missing values. It creates new values to replace the ones that have been lost. In addition, when making the classification, it calculates and applies the relevance level of the variables.

However, it has a complex structure since it is made up of multiple decision trees. Another problem is that it is difficult to comprehend how it works.

3) *Decision Tree (ID3)*: The divide-and-conquer technique is used in the decision trees method. It condenses enormous amounts of worthless data and groups them into a meaningful whole. As a result, it's commonly utilized for categorizing huge and complicated datasets [8].

Unfortunately, overfitting occurs as a result of these complicated trees, preventing fair findings. Another drawback is that a minor misrepresentation of the data at the start of the decomposition process might result in distinct branches and misleading findings. To deal with this, the data that will be used must normally be pre-processed.

4) *Adaboost [9]*: AdaBoost (Adaptive Boosting) is a machine learning technique that was created to increase classification performance. The following is the core operating concept of boosting algorithms: With preliminary draft guidelines, the data is initially separated into groups. New rules are added to this preliminary draft rule every time the algorithm is performed. Many "basic rules," or rules with minimal performance, are obtained this way.

These weak rules are integrated into a single rule that is considerably stronger and more successful after the algorithm has been tested numerous times. The Algorithm applies a weighted coefficient to each weak rule during this process, with the greatest coefficient value going to the lowest mistake rate. When final regulations are chosen, these weight values come into effect. The final rule is made by prioritizing the low-scoring weak regulations.

Few of the benefits of using Adaboost is that, in this approach, there is no need to apply the variable transformation. It has the ability to conduct operations on an excessive number of weak rules. In AdaBoost, the problem of overfitting is quite rare. It's also a good idea to deal with the data set's missing values.

On the other hand, it can be mentioned as a disadvantage that it is weak against noise and extreme values and that the predictive value does not have the highest values when compared to other algorithms.

5) *kNN*: In this case, KNN uses training data of known class types to identify the class of fresh data that cannot be classified. This is determined by monitoring the new

sample's closest neighbors, for which no classifications are given. KNN, which performs well on multidimensional data and is a quick algorithm during the training phase, is extremely slow during the estimation step [8].

C. Adversarial Machine Learning [10]

Machine Learning systems and models are currently being implemented in every domain imaginable, resulting in broad use of software-based inference and decision-making. Although it is becoming more clear that machine learning presents new vulnerabilities in software systems, the technical community's knowledge of the nature and scope of these flaws remains restricted.

Detection and monitoring systems in the security area today ingest massive volumes of data and extract actionable information that would have been unattainable in the past. Despite these remarkable advancements, the technical community's awareness of the vulnerabilities inherent in the design of systems based on machine learning, as well as the ways to guard against them, is still in its early stages.

1) *Attack on Decision Tree Algorithm [2]*: The input domain is recursively partitioned to construct decision trees. Partitioning is done by choosing a feature and a condition threshold that minimizes a cost function over the training data the best. Each node is an if-else statement that includes a threshold condition that corresponds to one of the sample's characteristics. A sample is categorised by traversing the decision tree from root to leaf according to conditions provided in intermediary tree nodes. The class allocated is indicated by the leaf reached.

An attack on such a classifier has been implemented by IBM's ART toolbox [11] based on the following intuition: Adversaries can potentially create adversarial inputs that decision trees misclassify. As per our discovery, this is the first adversarial sample crafting approach for decision trees that has been proposed. The notion takes use of the classifier model's underlying tree structure. Given a sample and a tree, the authors simply look for leaves with different classes in the vicinity of the leaf corresponding to the decision tree's initial forecast for the sample to locate an adversarial sample. The sample is then modified according to the requirements on the route from the original leaf to the adversarial leaf, forcing the decision tree to misclassify the sample in the adversarial class defined by the newly recognized leaf.

D. Related Work [12]

1) *Fitni and Ramli [13] (Implementation of ensemble learning and feature selection for performance improvements in anomaly-based intrusion detection systems)*: This study used an ensemble model to examine seven single learners in order to determine the best performers for inclusion into a classifier unit. The following are the seven classifiers: Random Forest, Gaussian Naive Bayes, Decision Tree, Quadratic Discriminant Analysis (QDA), and Gradient Boosting stands for Logistic Regression. Python and Scikit-learn were used to create the models. Samples with missing values and infinity were eliminated during preprocessing. Records that were duplicates of the header rows were also deleted. In an 80-20 ratio, the dataset was then divided into

training and testing validation sets.

Using Spearman's rank correlation coefficient and the Chi-squared test, 23 characteristics were selected for feature selection, a strategy for picking the most essential features of a predictive model. Gradient Boosting, Logistic Regression, and QDA emerged as the top performers for use in the ensemble model after examination of the seven learners using these characteristics. This model has accuracy, precision, and recall scores of 98.80%, 98.80%, and 97.10%, respectively, as well as an Area Under the Receiver Operating Characteristic Curve (AUC) of 0.94.

2) *Huancayo Ramos et al. [14] (Benchmark-based reference model for evaluating Botnet detection tools driven by traffic-flow analytics)*: To identify the best botnet classifier, five learners were assessed on two datasets (CSE-CIC-IDS-2018 and ISOT HTTP Botnet). DNS traffic from both malicious and benign sources may be seen in the ISOT HTTP Botnet dataset. Random Forest, Decision Tree, k-NN, Naive Bayes, and SVM are among the study's learners. Various strategies were used to choose features, including Random Forest's feature significance method. CICIDS2018 had 19 independent characteristics after feature selection, whereas ISOT HTTP had 20, with destination port number, source port number, and transport protocol among the features chosen. Python and Scikit-learn were used to create the models.

A training set including 80% of the botnet instances was subjected to five-fold cross-validation. The remaining botnet instances were used as a test set. The Grid Search algorithm [80] was utilized for optimization. In CICIDS2018, the Random Forest and Decision Tree classifier achieved a 99.99 percent accuracy rate. Precision was 100 percent and recall was 99.99 percent for both learners when tied to this accuracy. The Random Forest and Decision Tree classifier likewise had the greatest ISOT HTTP accuracy (99.94 percent for Random Forest and 99.90 percent for Decision Tree). The lack of information on data preparation is one of the paper's limitations.

3) *Karatas et al. [15] (Increasing the performance of machine learning-based IDSs on an imbalanced and up-to-date dataset)*: The authors assessed the performance of six learners on CSE-CIC-IDS-2018 using the Synthetic Minority Oversampling Technique (SMOTE) technique to solve class imbalance. k-NN, Random Forest, Gradient Boosting, Adaboost, Decision Tree, and Linear Discriminant Analysis were the classifiers used. Keras, TensorFlow, and Scikit-learn were used to create the learners in a Python environment. CICIDS2018, according to the authors, has around 5,000,000 samples. However, because the whole dataset comprises about 16,000,000 cases, the authors should make it explicit that a subset was used. Missing values and "Infinity" were removed from the dataset during preprocessing.

Rows were also shuffled for unpredictability, and one-hot encoding was utilized. A training set containing 80% of the occurrences was subjected to five-fold cross-validation. The test set was made up of the remaining occurrences. The entire dataset size rose by 17% when SMOTE was applied. With an accuracy of 99.69 percent, precision and recall scores of

99.70 percent and 99.69 percent, respectively, the Adaboost learner was found to be the best performer. This study, in our opinion, should have gone into further detail on data cleansing. Nonetheless, this publication has done a good job of covering data cleansing among the surveyed papers.

4) *Giovanni et al. [16] Evaluating the effectiveness of Adversarial Attacks against Botnet Detectors*: The authors rely on 4 recent, public and labelled datasets of network traffic that include communications generated by botnet-related pieces of malware: CTU-13 [17], CIC-IDS 2018 [7], UNB-CA Botnet6 [18]. Multiple botnet detectors were used by the authors for tests, each based on a different machine learning classifier from the list below: Random Forest (RF), Decision Trees (DT), AdaBoost (AB), Multi-Layer Perceptron (MLP), K-Nearest Neighbor (KNN), Gradient Boosting (GB), Linear Regression (LR), Support Vector Machines (SVM), Random Forest (RF), Decision Trees (DT), AdaBoost (AB), Multi-Layer Perceptron (MLP), K-Nearest Neighbor (KNN), Gradient Boosting (GB), Naive Bayes (NB), ExtraTrees (ET), Bagging (Bag), Stochastic Gradient Descent Linear Classifier (SGD).

Next, the authors trained each classifier on each unique botnet family in each dataset since each dataset may contain several botnet families. The authors create a huge number of detectors to see how different machine learning classifiers, trained in various network contexts, respond to adversarial attacks. Each detector is independently trained, verified, and assessed. They employ sets of features from relevant research on flow-based classifiers to train each classifier. They employ 80% of each botnet family's accessible malicious samples for training and 20% for testing; benign samples are picked at random to construct sets with a benign-to-malicious samples ratio of 90:10. Each classifier is tested using 3-fold cross validation after executing many grid-search operations to identify its best configuration settings. Precision, Detection Rate (DR, or Recall), and F1-score are used to evaluate each detector's performance.

For adversarial settings, the authors generate adversarial sample by evaluating matching malicious samples for each detector (which is associated with a certain botnet family in a given dataset) and alter their values at random. Up to 4 separate features can be perturbed: flow duration, transmitted bytes, received bytes, and exchanged packets; the changes are made in tiny increments of [1..120] for flow duration (in seconds) and [1..1024] for the remaining features. Moreover, the related derived features were updated to preserve consistency (e.g.: bytes per second). The Recall of the more robust ML-detector (based on the MLP algorithm) decreases from 0.932 to 0.597 for the UNB-CA Botnet dataset, while the Recall of the most impacted ML-detector (based on SVM) goes from 0.914 to 0.198. For the CTU-13 dataset, the Attack Severity is significantly worse: the Recall for the less impacted ML-detection method (based on RF) goes from 0.967 to 0.439.

III. CONCLUSIONS

Use of Machine Learning mechanism in Network-based Anomaly detection systems has been studied to solve a

problem of security. However, the the attack surface introduced by machine learning on such systems has hardly been investigated, which our work tries to achieve. Our work provides a natural structure for debating the consideration of vulnerabilities in the algorithm employed behind Network-based anomaly detection systems in their threat models.

For the anomaly detection challenge, it was observed that Decision Trees and Adaboost classifiers function well (Table III). Whereas, Naive Bayes model performs particularly poorly because this classifier assumes that all data points are independent. However, because the entries in this situation are not independent of one another, the Naive Bayes classifier performs badly.

Accuracy	Naive Bayes	Random Forest	ID3	Adaboost	MLP	kNN
DDoS attack-HOIC	0.606	0.973	0.994	0.993	0.766	0.992
DDoS attacks-LOIC-HTTP	0.701	0.989	0.987	0.990	0.700	0.994
DoS attacks-Hulk	0.337	0.982	0.982	0.982	0.980	0.978
Bot	0.589	0.991	0.995	0.993	0.996	0.996
FTP-BruteForce	0.692	0.983	0.983	0.983	0.983	0.983
SSH-BruteForce	0.557	0.990	0.990	0.990	0.919	0.990
Infiltration	0.358	0.747	0.747	0.746	0.721	0.730
DoS attacks-SlowHTTPTest	0.696	0.977	0.977	0.977	0.956	0.973
DoS attacks-GoldenEye	0.369	0.960	0.974	0.972	0.706	0.981
DoS attacks-Slowloris	0.337	0.991	0.993	0.996	0.890	0.991
Web attacks	0.680	0.918	0.921	0.930	0.443	0.92

TABLE III

Accuracy of ML Models for different attack types

Accuracy	Normal	Adversarial
DDoS attack-HOIC	0.993	0.710
DDoS attacks-LOIC-HTTP	0.987	0.985
DoS attacks-Hulk	0.982	0.965
Bot	0.995	0.701
FTP-BruteForce	0.983	0.706
SSH-BruteForce	0.990	0.704
Infiltration	0.747	0.646
DoS attacks-SlowHTTPTest	0.977	0.701
DoS attacks-GoldenEye	0.974	0.702
DoS attacks-Slowloris	0.993	0.563
Web attacks	0.921	0.243

TABLE IV

Comparison of accuracy under the normal and adversarial settings.

From our findings listed in Table IV, it can be observed that the prediction accuracy for most attacks reduces dramatically in the adversarial setting. Moreover, we draw the following conclusions that are supported by outcomes of our work:

- For identifying network-based attacks, the Decision tree and Adaboost algorithms are the most accurate. These methods can thus be used in anomaly detection systems.
- Adversaries can target machine learning models to lower the accuracy of network-based attack predictions.
- This implies that an attacker can launch a network-based attack that the anomaly detection system will miss.
- Machine learning approaches are not very resistant to adversarial attacks, necessitating the use of additional methods for anomaly detection or using defense mechanisms against machine learning models to make them more resilient.

In this work, we performed an attack against the best

ML algorithm concluded by comparing the accuracy. This work can be extended to attacking other machine learning techniques using adversarial transferability. Our contribution sets the door for the development of much-needed defensive techniques for machine learning based anomaly detectors on network traffic in adversarial circumstances.

IV. WORK DISTRIBUTION

Tasks	Owner
Literature Survey	Aniket and Rishabh
Dataset Selection	Rishabh
Design Process	Aniket
Machine Learning Training	Rishabh
Implementing Adversarial ML	Aniket
PPT	Aniket and Rishabh
Report	Aniket and Rishabh

TABLE V

REFERENCES

- [1] A. Khraisat, I. Gondal, P. Vamplew, J. Kamruzzaman, "Survey of intrusion detection systems: techniques, datasets and challenges", *Cybersecurity*, vol. 2, no. 1, p. 20, Jul 2019, doi:10.1186/s42400-019-0038-7, URL: <https://doi.org/10.1186/s42400-019-0038-7>.
- [2] N. Papernot, P. McDaniel, I. Goodfellow, "Transferability in Machine Learning: from Phenomena to Black-Box Attacks using Adversarial Samples", , 2016, doi:10.48550/ARXIV.1605.07277, URL: <https://arxiv.org/abs/1605.07277>.
- [3] D. Datasets, "MIT Lincoln Laboratory, DARPA Intrusion Detection Evaluation Data Sets", , 1998.
- [4] H. S., "KDD Cup 1999 Data", *The UCI KDD Archive*, 1999.
- [5] M. Tavallaee, E. Bagheri, W. Lu, A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set", in *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*, pp. 1–6, 2009, doi:10.1109/CISDA.2009.5356528.
- [6] "Toward developing a systematic approach to generate benchmark datasets for intrusion detection", *Computers & Security*, vol. 31, no. 3, pp. 357–374, 2012, doi:<https://doi.org/10.1016/j.cose.2011.12.012>.
- [7] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization.", *ICISSp*, vol. 1, pp. 108–116, 2018.
- [8] S. B. Kotsiantis, I. Zaharakis, P. Pintelas, *et al.*, "Supervised machine learning: A review of classification techniques", *Emerging artificial intelligence applications in computer engineering*, vol. 160, no. 1, pp. 3–24, 2007.
- [9] R. E. Schapire, "The boosting approach to machine learning: An overview", *Nonlinear estimation and classification*, pp. 149–171, 2003.
- [10] N. Papernot, P. McDaniel, A. Sinha, M. P. Wellman, "SoK: Security and Privacy in Machine Learning",

- in *2018 IEEE European Symposium on Security and Privacy (EuroS P)*, pp. 399–414, 2018, doi: 10.1109/EuroSP.2018.00035.
- [11] M.-I. Nicolae, M. Sinn, M. N. Tran, B. Buesser, A. Rawat, M. Wistuba, V. Zantedeschi, N. Baracaldo, B. Chen, H. Ludwig, I. Molloy, B. Edwards, “Adversarial Robustness Toolbox v1.2.0”, *CoRR*, vol. 1807.01069, 2018, URL: <https://arxiv.org/pdf/1807.01069>.
 - [12] J. L. Leevy, T. M. Khoshgoftaar, “A survey and analysis of intrusion detection models based on CSE-CIC-IDS2018 Big Data”, *Journal of Big Data*, vol. 7, no. 1, p. 104, Nov 2020, doi:10.1186/s40537-020-00382-x, URL: <https://doi.org/10.1186/s40537-020-00382-x>.
 - [13] Q. R. S. Fitni, K. Ramli, “Implementation of Ensemble Learning and Feature Selection for Performance Improvements in Anomaly-Based Intrusion Detection Systems”, in *2020 IEEE International Conference on Industry 4.0, Artificial Intelligence, and Communications Technology (IAICT)*, pp. 118–124, 2020, doi:10.1109/IAICT50021.2020.9172014.
 - [14] K. S. Huancayo Ramos, M. A. Sotelo Monge, J. Maestre Vidal, “Benchmark-Based Reference Model for Evaluating Botnet Detection Tools Driven by Traffic-Flow Analytics”, *Sensors*, vol. 20, no. 16, 2020, URL: <https://www.mdpi.com/1424-8220/20/16/4501>.
 - [15] G. Karatas, O. Demir, O. K. Sahingoz, “Increasing the Performance of Machine Learning-Based IDSs on an Imbalanced and Up-to-Date Dataset”, *IEEE Access*, vol. 8, pp. 32150–32162, 2020, doi: 10.1109/ACCESS.2020.2973219.
 - [16] G. Apruzzese, M. Colajanni, M. Marchetti, “Evaluating the effectiveness of Adversarial Attacks against Botnet Detectors”, in *2019 IEEE 18th International Symposium on Network Computing and Applications (NCA)*, pp. 1–8, 2019, doi: 10.1109/NCA.2019.8935039.
 - [17] S. Garcia, M. Grill, J. Stiborek, A. Zunino, “An empirical comparison of botnet detection methods”, *computers & security*, vol. 45, pp. 100–123, 2014.
 - [18] E. B. Beigi, H. H. Jazi, N. Stakhanova, A. A. Ghorbani, “Towards effective feature selection in machine learning-based botnet detection approaches”, in *2014 IEEE Conference on Communications and Network Security*, pp. 247–255, IEEE, 2014.